

What Is an MCP Server (and How Do I Use One in Claude Code)?

A step-by-step tutorial for complete beginners

```
You (typing a prompt)
|
v
Claude Code -----> "I need GitHub info to answer this"
(the HOST)
|
v
MCP Client (built into Claude Code)
| speaks the MCP protocol
v
MCP Server (a small program that knows how to talk to GitHub)
|
v
The real system (GitHub's API)
```

Worked examples inside: **Context7** · **GitHub** · **Figma**

Now with: bring-your-own API keys · connecting servers from the Claude Code CLI

How this works: Read Parts 1–3 to understand *what* MCP is and *where* it lives in Claude Code. Then follow the three worked examples (Context7, GitHub, Figma) step by step — each one is a real server you can add and try today. Each example now ends with a subsection on connecting it with *your own* credentials.

Before you start: You need Claude Code installed and a terminal open. If you can type `claude` and see a prompt, you're ready. Exact install commands for specific MCP servers change over time — if a command below doesn't work, check the server's own docs for the current one; the *concepts* (add → authenticate → verify → use) stay the same.

PART 1

What is MCP, in one paragraph

MCP (Model Context Protocol) is an open standard that lets an AI application — the **host** — talk to outside tools and data sources through one common interface, instead of needing a hand-built integration for every single tool. Claude Code is a host. Before MCP, if you wanted Claude to read your GitHub issues *and* check your Figma designs *and* pull live library docs, someone would need to write three separate custom integrations. With MCP, each of those systems just needs to expose **one MCP server**, and any MCP-compatible host (Claude Code, Claude Desktop, other AI apps) can talk to it the same way.

Think of MCP as **"USB-C for AI tools"**: one standard plug, many devices.

PART 2

The mental model

```
You (typing a prompt)
  |
  v
Claude Code -----> "I need GitHub info to answer this"
(the HOST)
  |
  v
MCP Client (built into Claude Code)
  | speaks the MCP protocol
  v
MCP Server (a small program that knows how to talk to GitHub)
  |
  v
The real system (GitHub's API)
```

- **Host** — the AI application you're using. Here, that's Claude Code.
- **MCP Client** — the part of the host that manages the connection to a server. You never interact with this directly.

- **MCP Server** — a program that exposes a specific system's capabilities (tools, data, prompts) in the MCP format. Someone else usually builds and maintains this (Anthropic, GitHub, Figma, a community author, or you).
- **The real system** — the actual API, database, or app the server is a wrapper around.

When Claude decides it needs something an MCP server can provide, it calls one of that server's **tools** (like a function), gets the result back, and keeps reasoning with that information — same as any other tool call you've seen Claude Code make.

PART 3

Local vs. remote servers

MCP servers come in two flavors. You'll meet both in the worked examples below.

Type	How it runs	Example	Auth
Local (stdio)	Claude Code launches it as a child process on your machine (often via <code>npx</code>)	A docs-lookup server, a local file-indexer	Usually none, or an API key you provide once
Remote (HTTP/SSE)	A server hosted elsewhere; Claude Code connects over the network	GitHub's official MCP server	Usually OAuth login or a personal access token

Gotcha: a remote server means your prompts and Claude's tool calls travel to that server's infrastructure, not just your machine. Only add servers you trust.

A note on transports: HTTP (sometimes written `streamable-http`) is the current standard for remote servers. The older `--transport sse` still works, but Anthropic now marks it deprecated — prefer `--transport http` whenever a server offers it.

PART 4

Where MCP config lives in Claude Code

You manage servers with the `claude mcp` command family, run from your terminal (outside an active session):

```
claude mcp add <name> <command-or-url>    # add a server
claude mcp list                            # see everything configured
claude mcp get <name>                      # inspect one server
claude mcp remove <name>                   # remove a server
```

Every server is added at a **scope**, which controls who can see it:

Scope flag	Stored where	Use it for
<code>--scope local</code> (default)	Your personal Claude Code settings for this project only (<code>~/.claude.json</code>)	Servers only you use, with your own keys
<code>--scope project</code>	<code>.mcp.json</code> in the project root (commit it!)	Shared team servers everyone on the repo should have
<code>--scope user</code>	Your global Claude Code settings	Servers you want in <i>every</i> project on your machine

Once inside a Claude Code session, type `/mcp` at any time. This shows every connected server, its status, and — for servers needing OAuth — lets you log in interactively.

PART 5 — WORKED EXAMPLE 1

Context7 (live library docs)

What it does: Context7 fetches current, version-accurate documentation for libraries and frameworks, so Claude isn't relying on stale training data when it writes code against an API.

Add it (local, no auth needed)

```
claude mcp add context7 -- npx -y @upstash/context7-mcp
```

Verify it's connected

```
claude mcp list
```

or, inside a session, type `/mcp` and confirm `context7` shows as connected.

Use it in a prompt

Just mention the library, or say so explicitly:

```
Using context7, show me how to set up a basic route
in the latest version of FastAPI.
```

Claude calls Context7's tools to pull the current FastAPI docs, then writes the code against what's actually current — instead of guessing from memory.

Using your own API key

Context7 is the odd one out: it works with **no credentials at all**. A key doesn't unlock the server — it raises your rate limits and attaches usage to your account instead of your IP address. Add one when you start seeing 429s.

Where to get it

1. Go to context7.com/dashboard and create a free account.
2. In the **API Keys** card, click **Create API Key**.
3. Name it something you'll recognize later — "Claude Code", "work laptop".
4. Copy it immediately. The key begins with `ctx7sk-` and is shown **once**.

Which scopes to select

None to choose. Context7 keys have no scope selector — the key identifies your account and nothing more. There is no minimum to get wrong here, which is exactly why this is the easiest of the three to set up.

The config, with the key as an env var

Set the variable in your shell profile once:

```
export CONTEXT7_API_KEY="ctx7sk-your-key-here"
```

Then add the server. The local server reads `CONTEXT7_API_KEY` from its own environment, so you pass no key on the command line at all:

```
claude mcp add --env CONTEXT7_API_KEY=${CONTEXT7_API_KEY} --transport stdio context7 \
-- npx -y @upstash/context7-mcp
```

For the remote server, put the variable in the header and let Claude Code expand it at connection time. **Use single quotes** — double quotes make your shell substitute the real key into the stored config, which is exactly what you're trying to avoid:

```
claude mcp add --transport http context7 https://mcp.context7.com/mcp \
--header 'CONTEXT7_API_KEY: ${CONTEXT7_API_KEY}'
```

What breaks without it: nothing, at first — then tool calls start returning "Rate limited. Monthly quota exceeded" and Claude quietly falls back to guessing from memory. A *wrong* key is louder: "Unauthorized. Please check your API key."

PART 6 — WORKED EXAMPLE 2

GitHub (repos, issues, PRs)

What it does: Lets Claude read and act on your GitHub repos — list issues, inspect a PR's diff and comments, search code, open issues — using your GitHub identity and permissions.

Add it (remote server, needs auth)

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
```

Authenticate

Start (or re-enter) a Claude Code session and run:

```
/mcp
```

Select the `github` server and follow the OAuth prompt — it opens your browser, you approve access, and Claude Code stores the resulting token. You only need to do this once per machine (per scope).

Use it in a prompt

```
List the open issues in this repo labeled "bug"
and summarize the two oldest ones.
```

```
What changed in pull request #42, and are there
any unresolved review comments?
```

Gotcha: the GitHub server only sees what your GitHub account can see. If a repo/PR is private and you don't have access, neither does Claude.

Using your own fine-grained token

Above, you used the OAuth flow. The alternative is a **fine-grained personal access token (PAT)**, which you'll prefer when you want to name the exact repositories Claude can touch — OAuth grants what the app asks for, a fine-grained PAT grants what *you* list.

Where to get it

1. GitHub → your avatar (top right) → **Settings**
2. Left sidebar, at the bottom → **Developer settings**
3. **Personal access tokens** → **Fine-grained tokens** → **Generate new token**
4. Direct link: <https://github.com/settings/personal-access-tokens>

Which permissions, and why the minimum is enough

Field	Set it to	Why
Resource owner	You, or the org that owns the repos	An org owner may need to approve the token before it works
Repository access	Only select repositories → pick them	The blast radius of a leaked token is exactly this list
Metadata	Read-only	Auto-selected; nearly every endpoint requires it
Contents	Read-only	Reading code and files
Issues	Read-only	Listing and reading issues
Pull requests	Read-only	Reading a PR's diff and review comments

Those four permissions cover everything the prompts above do — they all *read*. Move a permission to **Read and write** only at the moment you want Claude to open an issue or push a branch, and only for that repo. Note that a fine-grained token always carries read-only access to public repositories regardless of what you select.

The config, with the token as an env var

```
export GITHUB_MCP_PAT="github_pat_your_token_here"
```

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header 'Authorization: Bearer ${GITHUB_MCP_PAT}'
```

Single quotes again — that stores the literal `${GITHUB_MCP_PAT}` in your config, and Claude Code resolves it from your environment on every connection. Your config file stays safe to read over someone's shoulder.

Gotcha: `claude mcp add` writes the config without ever contacting GitHub, so a typo'd or empty token is accepted here and only fails later, at connection time. Always follow it with `claude mcp list`.

What breaks with a bad token: `claude mcp list` shows `✗ Failed to connect` with a `401` in the failure detail. What breaks with a *valid token missing a repo* is sneakier — the server connects fine and simply reports the repo as not found, indistinguishable from a repo that doesn't exist. If Claude insists your repo isn't there, check the token's repository list before you check your spelling.

Figma (design files)

What it does: Lets Claude read a Figma file's structure directly — frames, components, layout, styles — so it can translate a design into code instead of you describing it by hand.

Add it

Figma ships a desktop MCP server that runs locally from the Figma desktop app. Steps:

1. Open the Figma desktop app and open a Figma Design file.
2. In the toolbar at the bottom, switch to **Dev Mode** (or press `Shift D`).
3. In the **MCP server** section of the inspect panel, click **Enable desktop MCP server**.
4. The server runs at `http://127.0.0.1:3845/mcp`. Add it:

```
claude mcp add --transport http figma-desktop http://127.0.0.1:3845/mcp
```

Changed since the last edition: earlier versions of this tutorial used `--transport sse` with a `/sse` path. Figma's desktop server has moved to streamable HTTP at `/mcp`, and Anthropic has deprecated the SSE transport. Use the command above. Figma controls this server, not Anthropic — if the add command fails, open Figma's own MCP docs and copy the URL it displays for your installed version.

Use it in a prompt

Select a frame or component in Figma first, then:

```
Using the selected Figma frame, generate the HTML
and CSS for this component.
```

Claude reads the real layout data (spacing, colors, typography tokens) from Figma instead of guessing from a screenshot.

Using your own credentials

Figma is the exception worth understanding, because the obvious answer is wrong. **Figma's official MCP server does not take an API key or a personal access token.** Both the remote server and the desktop server authenticate through your signed-in Figma session — remote via OAuth in the browser, desktop via the app you're already logged into. There is no header to set.

Remote server (Figma's recommended option)

```
claude mcp add --transport http figma https://mcp.figma.com/mcp
```

Then run `/mcp` inside a session, select **figma**, and choose **Authenticate**. Your browser opens; click **Allow Access**. Claude Code stores and refreshes the token for you. Nothing to paste, nothing to rotate.

So where does the Figma personal access token come in?

The Figma **PAT** authenticates the Figma *REST API*, not the official MCP server. It is what community servers such as Framelink (`figma-developer-mcp`) use. Reach for it when you're on a plan or seat that can't use the official server, or when you want a stdio server you fully control.

Where to get the PAT

1. Log in to Figma; from the file browser, open the account menu (top-left) → **Settings**
2. Select the **Security** tab
3. Under **Personal access tokens**, click **Generate new token**
4. Set an **expiration** and pick **scopes**
5. Click **Generate token** — copy it now, it is shown once

Which scopes, and why the minimum is enough

Scope	Take it?	Why
<code>file_content:read</code>	Yes	Reads the node tree, layout, styles, variables — this <i>is</i> the design-to-code use case
<code>library_content:read</code>	Only if needed	Published components and styles from a shared library
<code>file_comments:read</code>	No	Reading a frame's structure needs nothing from comments
<code>files:read</code>	Avoid	Figma has deprecated it in favour of the narrower scopes above

`file_content:read` alone covers "turn this frame into HTML and CSS." Everything else is surface area you'd be granting for no return. Non-expiring tokens can no longer be created, so pick the shortest expiry you can live with.

The config, with the token as an env var

```
export FIGMA_API_KEY="figd_your_token_here"
```

```
claude mcp add --env FIGMA_API_KEY=${FIGMA_API_KEY} --transport stdio figma-rest \  
-- npx -y figma-developer-mcp --stdio
```

What breaks: with the official desktop server, a connection refused on port 3845 means the toggle in Figma is off or the app is closed — it is never a credential problem. With the Framelink server and a bad PAT, tool calls return a `403` from the Figma API; with a valid PAT missing `file_content:read`, the server connects and then fails on the first real read.

PART 8

Verifying and managing servers

Command	What it does
<code>/mcp</code> (inside a session)	Lists connected servers, shows status, starts OAuth login
<code>claude mcp list</code>	Lists configured servers from the terminal, with health status
<code>claude mcp get <name></code>	Shows one server's full config
<code>claude mcp remove <name></code>	Removes a server

The first time a server's tool is about to run, Claude Code shows a permission prompt describing exactly what it's about to do. Review it — this is your checkpoint before an MCP server touches anything.

PART 9

Troubleshooting

Symptom	Likely cause	Fix
Server doesn't show up in <code>/mcp</code>	Wrong scope, or added from a different project folder	Run <code>claude mcp list</code> from the project root; re-add with the right <code>--scope</code>
"Authentication required" keeps reappearing	OAuth token expired or was never completed	Run <code>/mcp</code> , select the server, redo the login flow
<code>npm</code> command fails on add	Node.js not installed, or package name/version changed	Confirm <code>node -v</code> works; check the server's current install command in its own docs
Claude never calls the tool	It didn't recognize the request needs that server	Be explicit: name the server or the exact action ("using GitHub, ...")

Quick-reference cheat sheet

Server	Type	Add command	Auth	Try this prompt
Context7	Local (stdio)	<code>claude mcp add context7 -- npx -y @upstash/context7-mcp</code>	None	"Using context7, show the current React useEffect API."
Context7 <i>with key</i>	Local (stdio)	<code>claude mcp add --env CONTEXT7_API_KEY=\${CONTEXT7_API_KEY} --transport stdio context7 -- npx -y @upstash/context7-mcp</code>	API key (raises rate limits)	Same, but you stop hitting 429s
GitHub	Remote (HTTP)	<code>claude mcp add --transport http github https://api.githubcopilot.com/mcp/</code>	OAuth via / mcp	"List open issues labeled bug."
GitHub <i>with PAT</i>	Remote (HTTP)	<code>claude mcp add --transport http github https://api.githubcopilot.com/mcp/ --header 'Authorization: Bearer \${GITHUB_MCP_PAT}'</code>	Fine-grained PAT	Same, scoped to repos you chose
Figma <i>desktop</i>	Local (HTTP, via Figma desktop)	<code>claude mcp add --transport http figma-desktop http://127.0.0.1:3845/mcp</code>	Figma desktop login	"Generate CSS from the selected frame."
Figma <i>remote</i>	Remote (HTTP)	<code>claude mcp add --transport http figma https://mcp.figma.com/mcp</code>	OAuth via / mcp	"Implement the design at this Figma link."
Figma <i>via REST</i>	Local (stdio)	<code>claude mcp add --env FIGMA_API_KEY=\${FIGMA_API_KEY} --transport stdio figma-rest -- npx -y figma-developer-mcp --stdio</code>	Figma PAT	"Fetch the layout metadata for this frame URL."

Connecting MCP Servers with Claude Code

Everything below is literal. Copy it, and change only the values that are yours.

11.1 — Add each server

```
# Context7 — local, keyless
claude mcp add --transport stdio context7 -- npx -y @upstash/context7-mcp

# Context7 — local, with your key from the environment
claude mcp add --env CONTEXT7_API_KEY=${CONTEXT7_API_KEY} --transport stdio context7 \
  -- npx -y @upstash/context7-mcp

# GitHub — remote, OAuth (no token needed; sign in with /mcp afterwards)
claude mcp add --transport http github https://api.githubcopilot.com/mcp/

# GitHub — remote, with your own fine-grained PAT
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header 'Authorization: Bearer ${GITHUB_MCP_PAT}'

# Figma — remote, OAuth (sign in with /mcp afterwards)
claude mcp add --transport http figma https://mcp.figma.com/mcp

# Figma — desktop server, requires the toggle enabled in the Figma app
claude mcp add --transport http figma-desktop http://127.0.0.1:3845/mcp
```

Two rules that will save you an afternoon:

- **All flags come before the server name.** `--transport`, `--env`, `--scope` and `--header` all go first.
- `--` separates Claude's flags from the server's own command. Everything after `--` is handed to the server untouched. Without it, Claude Code tries to parse the server's flags as its own.

11.2 — Passing secrets with `--env`

`--env` (short form `-e`) sets environment variables inside the **stdio server's** process. Use it for local servers. For remote servers there is no process to hand variables to — use `--header` instead.

```
# One variable
claude mcp add --env FIGMA_API_KEY=${FIGMA_API_KEY} --transport stdio figma-rest \
  -- npx -y figma-developer-mcp --stdio

# Several
claude mcp add --env API_KEY=${API_KEY} --env REGION=us-east-1 --transport stdio myserver \
  -- npx -y some-mcp-package
```

Gotcha: never put the server name directly after `--env`. The CLI reads the next word as another `KEY=value` pair and rejects it. Keep at least one other flag between them — `--transport stdio` does the job, as above.

The rule for secrets: reference, don't paste. In every command here the key lives in your shell environment, and only the *reference* is written to disk. Claude Code expands `${VAR}` at connection time inside `command`, `args`, `env`, `url` and `headers`. You can also write `${VAR:-default}` for a fallback. If a variable isn't set and has no default, the server still loads, but `claude mcp list` warns you and passes the unexpanded `${VAR}` text through — which then fails as a credential.

11.3 — Local, project, or user scope

Scope	Loads in	Shared?	Stored in
<code>local</code> (default)	Current project only	No	<code>~/.claude.json</code>
<code>project</code>	Current project only	Yes, via git	<code>.mcp.json</code> in project root
<code>user</code>	All your projects	No	<code>~/.claude.json</code>

```
claude mcp add --transport http github --scope local https://api.githubcopilot.com/mcp/
claude mcp add --transport http github --scope project https://api.githubcopilot.com/mcp/
claude mcp add --transport http context7 --scope user https://mcp.context7.com/mcp
```

When each is the right choice

- **Local** — you're trying a server out, or it's wired to something specific to this one project. This is the default because it's the safe default.
- **Project** — the whole team needs it. `.mcp.json` gets committed, so *never* commit a real key: put `${VAR}` in the config and let each teammate set the variable in their own shell. Claude Code asks each person for approval before running a project server the first time.
- **User** — you want it everywhere and never want to think about it again. Context7 is the classic case: current library docs are useful in every repo you open.

If the same name is defined in more than one scope, **local wins, then project, then user**. The winning entry is used whole — fields are not merged across scopes.

11.4 — Verify the connection

```
claude mcp list      # every server, with a health status
claude mcp get github # one server's full config
```

Read the status, not just the output:

- ✓ `Connected` — working.

- ! `Needs authentication` — added correctly, you just haven't signed in. Run `/mcp`, or `claude mcp login <name>` from the shell.
- ✗ `Failed to connect` — it tried and was refused. The failure detail on that line carries the HTTP status.
- `Pending approval` (shown with a pause icon) — a project-scoped server waiting for you. Run `claude` in the folder and approve it.

Inside a session, `/mcp` shows the same list plus per-server tool counts, and is where you run OAuth logins and toggle a server off without deleting it.

A successful `Added ...` message means only that the config was **written**. It is not a connection test. Always follow it with `claude mcp list`.

11.5 — Troubleshooting prompts

Paste these into Claude Code as-is when a server won't come up. Replace only the values in angle brackets.

Run ``claude mcp list`` and ``claude mcp get <server-name>`` in the terminal. Show me the raw output, then tell me which of these the status points to: not configured, not authenticated, or configured but refused by the server. Do not fix anything yet — just tell me which one it is.

My `<server-name>` server shows "Needs authentication". Walk me through re-authenticating it from the terminal, and tell me where the token is stored afterwards and how I would revoke it.

My `<server-name>` server shows "Failed to connect". Read its config with ``claude mcp get <server-name>``, and check specifically for: an unexpanded `${VAR}` in the headers or env, leading or trailing whitespace in any value, and whether the transport matches what the vendor's docs say. Report what you find without changing anything.

I added `<server-name>` from a different folder and now ``claude mcp list`` doesn't show it. Check which scope it landed in, explain why it's invisible from here, and give me the exact ``claude mcp remove`` and ``claude mcp add`` commands to move it to `<local|project|user>` scope instead.

``claude mcp add`` succeeded for `<server-name>` but the tools never get called. Check whether the server is connected and how many tools it exposes, then show me a prompt that names the server explicitly so I can confirm the tools work at all.

Gotcha worth knowing before you debug anything: Claude Code warns when a config value has hidden leading or trailing whitespace — almost always a token pasted with a trailing newline. It reports the field name without echoing the value, and it does **not** trim it for you. Edit the config yourself.

Recap

1. MCP is a standard plug between an AI host and outside tools/data.
2. Claude Code is the host; each integration is a small server you add once.
3. Add with `claude mcp add`, check with `/mcp`, manage with `claude mcp list` / `remove`.
4. Local servers run on your machine; remote servers live elsewhere and usually need login.
5. Each server authenticates differently: Context7 is keyless with an optional rate-limit key, GitHub takes a fine-grained PAT scoped to repos you choose, and Figma uses OAuth — its PAT belongs to the REST API, not the official MCP server.
6. Put secrets in environment variables and reference them with `${VAR}`. Never paste a key into a config file, especially a committed `.mcp.json`.
7. Once added, just ask for what you need in plain language — Claude reaches for the right server's tools on its own.